



Embedding Flaris in a C application

The Flaris programming language · flaris-lang.org · github.com/flaris-lang/flaris · flaris-lang.org/discord

Version 1.0.2.1 **Generated** 2026-09-01 **License** see repository

© 2026 SolidInvention AB — created and maintained by SolidInvention AB

Flaris ships as a library as well as a program. Link `libflaris` into your application and Flaris becomes its scripting layer: load scripts, call their functions from C, and drive the scheduler from your own loop.

This is the counterpart to the [FFI guide](#). FFI is *Flaris calling your C code*; this is *your C code calling Flaris*. Both use the same value type, `FfiObject`.

- [1. What you download](#)
- [2. Hello, host](#)
- [3. Shipping scripts with your application](#)
- [4. Lifecycle](#)
- [5. Configuration](#)
- [6. Loading code](#)
- [7. Calling into Flaris](#)
- [8. Values and ownership](#)
- [9. Driving the scheduler](#)
- [10. Errors and diagnostics](#)
- [11. Limits and what is not supported](#)
- [12. API reference](#)

1. What you download

Get the library for your platform from the [downloads page](#). Each archive unpacks to the same shape:

```
flaris-lib/
├── include/
│   ├── flaris.h      # the embedding API – the only header you include
│   └── ffi_object.h  # FfiObject, the value type crossing the boundary
└── lib/              # the static and shared library
```

Two headers, and `flaris.h` is self-contained: it declares the whole API in terms of C types and `FfiObject`, so nothing about the VM's internals leaks into your build.

Platform	Static	Shared
macOS ARM64	<code>libflaris.a</code>	<code>libflaris.dylib</code>
Linux x86_64 / ARM64	<code>libflaris.a</code>	<code>libflaris.so</code>
Windows x64	<code>libflaris.a</code>	<code>flaris.dll + libflaris.dll.a</code>

This is the same VM the `flarism` command runs — compiler, runtime, JIT, standard builtins and debugger — without the command-line front end. Nothing in it calls `exit()`; your application stays in control.

Include path and link flags. Point your compiler at `include/`, link the library, and add the system libraries the VM needs:

```
# macOS
cc host.c flaris-lib/lib/libflaris.a -Iflaris-lib/include -o host \
    -lm -framework Security -framework CoreFoundation

# Linux
cc host.c flaris-lib/lib/libflaris.a -Iflaris-lib/include -o host \
    -lm -ldl -lpthread

# Windows (MSYS2 clang64)
cc host.c flaris-lib/lib/libflaris.a -Iflaris-lib/include -o host \
    -lws2_32 -lbcrypt -liphlpapi -lsecur32 -lbcrypt32
```

`flaris.h` is the only header you include; it pulls in `ffi_object.h` from beside it.

Linking the shared library instead. The shared library exports exactly what this document describes - the thirteen `Flaris*` functions plus `flarisConfigDefaults` - and nothing else; the VM's internals are hidden, so no future release can break you by moving one. On Windows, add `-DFLARIS_DLL` when you build against `flaris.dll`, so the declarations become `dllimport`:

```
cc host.c -DFLARIS_DLL flaris-lib/lib/libflaris.dll.a -Iflaris-lib/include -o host
```

macOS and Linux need no define, but they do need an `rpath`, or the executable will not find the library beside it at launch:

```
# macOS
cc host.c flaris-lib/lib/libflaris.dylib -Iflaris-lib/include \
    -Wl,-rpath,@executable_path -o host

# Linux
cc host.c flaris-lib/lib/libflaris.so -Iflaris-lib/include \
    -Wl,-rpath,'$ORIGIN' -o host
```

2. Hello, host

```
#include "flaris.h"
#include <stdio.h>

int main(void)
{
    FlarisInit(NULL);

    FlarisLoadSource("fn Greet(who: string): string {\n"
```

```
        return \"hello, \" + who;\n\"}\n\", \"<host>\");\n\nFfiObject args[1] = { ffi_string(\"world\") };\nFfiObject result;\nchar err[256];\n\nif (FlarisCall(\"Greet\", args, 1, &result, err, sizeof err) == FLARIS_OK)\n    printf(\"%s\\n\", AS_STRING(result));\nelse\n    printf(\"error: %s\\n\", err);\n\nFlarisFreeValue(&result);\nFlarisShutdown(0);\nreturn 0;\n}
```

```
$ ./host\nhello, world
```

3. Shipping scripts with your application

You can hand Flaris either source (`.fls`) or compiled bytecode (`.flx`). For anything you ship, prefer bytecode:

- it does not expose your script source,
- it skips compilation at startup,
- it is validated on load, so a corrupt or mismatched file is refused rather than half-run.

Compile with the `flarism` toolchain as part of your build:

```
flarism --compile game.flx game.flx
```

Then load it at runtime:

```
FlarisLoadFile(\"game.flx\");
```

A script you load this way is a **library of functions**, not a program. Give it either an `export` list or a `Main`, because `--compile` requires one of the two:

```
fn Greet(who: string): string { return \"hello, \" + who; }\nfn Update(dt: float): int     { /* ... */ return 0; }\n\nexport { Greet, Update };
```

Everything the script declares is callable by name afterwards, whether or not it appears in the `export` list — the list is there to satisfy the compiler and to document intent.

Bytecode is portable across platforms and machines: a `.flx` built on one target runs on any other.

4. Lifecycle

```
FlarisInit(&cfg)
|
|─ FlarisLoadFile("game.flx")    ← or FlarisLoadSource(...)
|
|─ FlarisCall(...)               ← as often as you like
|─ FlarisPump(0)                 ← if the script uses timers/fibers/IO
|
FlarisShutdown(0)
```

`FlarisShutdown` releases everything and leaves your process able to start a fresh VM, so reloading a script means shutting down and initialising again. It is idempotent: calling it twice, or without a matching init, does nothing.

5. Configuration

Pass `NULL` to `FlarisInit` for defaults, or fill in a `FlarisConfig`:

```
FlarisConfig cfg = flarisConfigDefaults;
cfg.installSignals = FLARIS_OFF;      /* keep your own signal handlers */
cfg.startIoPool    = FLARIS_OFF;      /* stay single-threaded          */
cfg.unsafe         = FLARIS_ON;       /* the script may use Ffi/Memory */
cfg.maxFibers      = 64;               /* power of two                  */
cfg.stackSize     = 8192;
cfg.libsPath       = "/opt/myapp/scripts";

if (FlarisInit(&cfg) != FLARIS_OK) {
    /* a value was rejected - nothing was initialised */
}
```

Switches

Every switch is tri-state: `FLARIS_DEFAULT` (which is `0`, so a zeroed struct is a valid starting point), `FLARIS_ON`, or `FLARIS_OFF`. Anything else is rejected rather than read as a default.

Switch	Default	What it does
<code>installSignals</code>	on	Installs the VM's <code>SIGINT</code> / <code>SIGSEGV</code> handlers. Off leaves yours alone — a fault inside the VM then reaches <i>you</i> .

Switch	Default	What it does
startIoPool	on	Starts the I/O worker threads. Off keeps your process single-threaded; I/O still completes, inline on the VM thread.
unsafe	off	Allows Ffi.*, raw Memory access and Buffer addresses. Off, those raise at runtime. This is the --unsafe flag.
jit	on	Native code generation.
optimizations	on	Constant folding and peephole passes.
debugInfo	on	Keeps line numbers and names, so stack traces are useful.
signatureChecks	on	Verifies a .flx fingerprint when one is pinned.
requireSigned	off	Refuses .flx not signed by a trusted key.
verbose	off	Emits the VM's own progress chatter at FLARIS_LOG_INFO.
colors	auto	ANSI colour in diagnostics. Irrelevant once you set a log handler.

unsafe is a trust decision. It lets a script load native code into your process via Ffi and read or write raw addresses via Memory. Leave it off for scripts you do not control.

Limits

Limit	Default	Range
stackSize	4096	65–65535 value-stack slots
fifoSize	1024	power of two, ≤ 1024
maxSlabs	1024	object-pool ceiling
maxFibers	256	power of two, ≤ 1024
maxFrames	64	8–1024 call frames per fiber
ioThreads	0	0 scales to the CPU count; ≤ 16
libsPath	none	Where import looks, ; -separated. Borrowed — must outlive the VM.

Every limit reads 0 as "keep the default", so you set only what you care about. An out-of-range value makes FlarisInit fail rather than being quietly clamped, and nothing is initialised when it does.

6. Loading code

```
FlarisLoadFile("game.flx");           /* bytecode – the shipping path */
FlarisLoadFile("game.flc");           /* source, compiled on load      */
FlarisLoadSource(text, "<host>");       /* source from memory           */
FlarisRunFile("program.flc");         /* ... and call Main(), returning its code */
```

FlarisLoadFile chooses by extension: .flx is loaded as bytecode, anything else is compiled as source.

All three loaders run the script's **top level** — global initialisers and function definitions — and stop. They do not call Main, and FlarisLoadSource does not require the script to have one. FlarisRunFile is the exception: it is about to call Main, so it insists on one and returns its exit code instead of exiting your process.

Load one script per VM. To swap scripts, call FlarisShutdown and initialise again.

7. Calling into Flaris

```
FfiObject args[2] = { ffi_string("player"), ffi_int(3) };
FfiObject result;
char err[256];

int rc = FlarisCall("Damage", args, 2, &result, err, sizeof err);
```

Return	Meaning
FLARIS_OK	result holds the return value
FLARIS_ERR_RAISED	the script threw; err holds "code: message"
FLARIS_ERR_NOTFOUND	no such function in the loaded script
FLARIS_ERR_NOT_FN	the name exists but is not a function
FLARIS_ERR_SUSPENDED	the function suspended (see below)
FLARIS_ERR_ARGS	bad argument count, or more than 32 arguments

`FlarisHasFunction("Damage")` reports whether a name is callable — useful when the script decides which hooks it implements:

```
if (FlarisHasFunction("OnPlayerJoin"))
    FlarisCall("OnPlayerJoin", args, 1, NULL, err, sizeof err);
```

A called function must run to completion. If it yields or awaits, the call returns `FLARIS_ERR_SUSPENDED`, because nothing above it can resume it. For work that suspends, have the script start a fiber and drive `FlarisPump` instead.

8. Values and ownership

Values cross the boundary as `FfiObject` — the same type FFI plugins use. Build them with the `ffi_*` constructors, read them with the `IS_*` / `AS_*` macros; both are documented in the [FFI guide](#) and declared in `ffi_object.h`.

```
ffi_int(42)           ffi_float(3.5)       ffi_bool(1)
ffi_string("text")    ffi_nil()            ffi_char('A')
ffi_array_n(3)        ffi_object_n(2)      ffi_table_n(rows, cols)
```

There are exactly two ownership rules.

FlarisCall consumes its arguments. Every argument is released by the call — on success and on every failure path alike, nested values included. Build them, hand them over, then neither free nor reuse them:

```
FfiObject args[1] = { ffi_string("owned by the call now") };
FlarisCall("Log", args, 1, NULL, err, sizeof err);
/* nothing to free – args[0] is gone */
```

It works this way because marshalling *adopts* the buffers inside an array or object argument. If you freed them as well, that would be a double free.

You free the result. `FlarisFreeValue(&result)` releases what a call produced, and is only ever for results:

```
FfiObject result;
if (FlarisCall("Report", NULL, 0, &result, err, sizeof err) == FLARIS_OK) {
    puts(AS_STRING(result));
    FlarisFreeValue(&result);
}
```

Scalars allocate nothing, so neither rule costs anything for them.

9. Driving the scheduler

Scripts that use timers, fibers or async I/O need scheduler time. You choose who owns the loop.

Your loop owns it. `FlarisPump` runs one non-blocking pass and returns how many fibers ran. It **never sleeps**, so your application decides its own cadence:

```
while (running) {
    FlarisPump(0);          /* 0 = every fiber that is ready right now */
    RenderFrame();
}
```

Bound the work per frame so a busy script cannot stall you:

```
FlarisPump(4);             /* at most four fibers this frame */
```

The VM owns it. `FlarisRunToCompletion` runs until nothing can become ready again, sleeping while fibers wait — what the `flarism` command does:

```
FlarisCall("Start", NULL, 0, NULL, err, sizeof err);
FlarisRunToCompletion();
```

`FlarisHasWork()` is true while a queued fiber, a live timer, a claimed event or an in-flight I/O operation could still make progress — the termination condition for a loop of your own:

```
while (FlarisHasWork()) {
    if (FlarisPump(0) == 0)
        SleepMilliseconds(1);    /* nothing was ready – your call how to idle */
}
```

Because `FlarisPump` never sleeps, a loop with no idle of its own can spin thousands of times before a 1 ms timer comes due. That is deliberate: the VM does not decide how long your application blocks.

10. Errors and diagnostics

Script errors come back as values

A script that throws does not print and vanish. The call returns `FLARIS_ERR_RAISED` and writes the exception's code and message into your buffer:

```
char err[256];
int rc = FlarisCall("Risky", NULL, 0, &result, err, sizeof err);

if (rc == FLARIS_ERR_RAISED)
    LogWarning("script error: %s", err);    /* e.g. "7: inventory full" */
```

`err` is always NUL-terminated and is cleared on entry, so a stale message never survives into a later call. The VM stays usable afterwards — the next call runs normally.

This holds for *any* failure inside the call, not just an explicit `throw`: a division by zero, an index out of bounds, or a blocked `Ffi` call with `unsafe` off all arrive the same way.

Nothing takes your process down

The library never calls `exit()`. An unhandled error inside a script is reported and unwound; your process keeps running and the VM stays usable. This is the one behaviour that differs from the `flarism` command, where exiting on an unhandled exception is the right thing to do.

Routing the VM's own output

By default the VM writes its diagnostics — runtime errors, warnings, compiler messages — to `stderr`. Hand it a callback instead:

```
static void MyLogger(int level, const char *message, void *userData)
{
    static const char *names[] = { "error", "warning", "info", "verbose" };
    fprintf(myLogFile, "[flaris %s] %s\n", names[level], message);
}

FlarisSetLogHandler(MyLogger, NULL);
```

`message` is formatted, NUL-terminated, and carries no trailing newline and no ANSI escapes; it is valid only for the duration of the call, so copy it if you keep it. Levels are `FLARIS_LOG_ERROR`, `FLARIS_LOG_WARNING`, `FLARIS_LOG_INFO` and `FLARIS_LOG_VERBOSE`. Pass `NULL` to go back to `stderr`.

You may set the handler before `FlarisInit`, so even startup diagnostics reach it. Compilation diagnostics go through it too, which is how you capture why a script failed to compile.

This covers what the *VM* says. What a *script* prints with `Console.WriteLine` still goes to `stdout`; redirect that by your own means if you need to.

11. Limits and what is not supported

One VM per process. VM state is process-wide, so there is no handle type and no way to hold two VMs at once. `FlarisShutdown` makes the process reusable, which covers reloading a script, but not isolating several scripts from each other.

One thread. Every function here must be called from the thread that called `FlarisInit`. Fibers are cooperative and run on that thread; the I/O worker threads never touch VM state.

One script per VM. Loading a second script into a live VM is not supported; shut down and initialise again.

No native modules. There is no way to register your own module so scripts can call `MyModule.Function(...)`. Defining one would mean building VM-internal structures by hand, which is not something a binary distribution can reasonably ask of you.

The supported way to give scripts access to your code is an **FFI plugin**: expose C functions from a shared library, and the script loads and calls them through `Ffi`. That path is documented, stable, and needs only `ffi_object.h` — the same header this API already uses for values. It requires `cfg.unsafe = FLARIS_ON`.

12. API reference

Everything below is declared in `flaris.h`.

Lifecycle

Function	Description
<code>int FlarisInit(const FlarisConfig *cfg)</code>	Start the VM. <code>NULL</code> means defaults. <code>FLARIS_OK</code> or <code>FLARIS_ERR_INIT</code> .
<code>void FlarisShutdown(int code)</code>	Release everything; idempotent. <code>code</code> only reaches an attached debugger.
<code>void FlarisSetArgs(int argc, char **argv)</code>	Make <code>argv</code> visible to <code>0s.Args</code> . Borrows <code>argv</code> .
<code>void FlarisSetLogHandler(FlarisLogFn fn, void *userData)</code>	Route VM diagnostics to <code>fn</code> ; <code>NULL</code> restores <code>stderr</code> .

Loading

Function	Description
<code>int FlarisLoadSource(const char *source, const char *name)</code>	Compile a string and run its top level. <code>name</code> labels diagnostics.
<code>int FlarisLoadFile(const char *path)</code>	Load a <code>.flx</code> , or compile a <code>.fls</code> , and run its top level.
<code>int FlarisRunFile(const char *path)</code>	Load and call <code>Main()</code> , returning its exit code.

Calling

Function	Description
<code>int FlarisCall(name, args, argc, outResult, errBuf, errSize)</code>	Call a function. Consumes <code>args</code> .
<code>int FlarisHasFunction(const char *name)</code>	1 if the name is callable.
<code>void FlarisFreeValue(FfiObject *value)</code>	Release a call result.

Scheduler

Function	Description
<code>int FlarisPump(int maxFibers)</code>	One non-blocking pass; returns fibers run. Never sleeps.
<code>int FlarisHasWork(void)</code>	True while anything could still make progress.
<code>void FlarisRunToCompletion(void)</code>	Run until quiescent, sleeping as needed.

Status codes

Constant	Value	Meaning
<code>FLARIS_OK</code>	0	Success
<code>FLARIS_ERR_NO_VM</code>	-1	No VM is running
<code>FLARIS_ERR_NOT_FN</code>	-2	The name exists but is not callable
<code>FLARIS_ERR_ARGS</code>	-3	Bad argument count
<code>FLARIS_ERR_RAISED</code>	-4	The script threw
<code>FLARIS_ERR_SUSPENDED</code>	-5	The function yielded or awaited
<code>FLARIS_ERR_INIT</code>	-10	A configuration value was rejected
<code>FLARIS_ERR_COMPILE</code>	-11	Script did not compile
<code>FLARIS_ERR_NOTFOUND</code>	-12	No such function